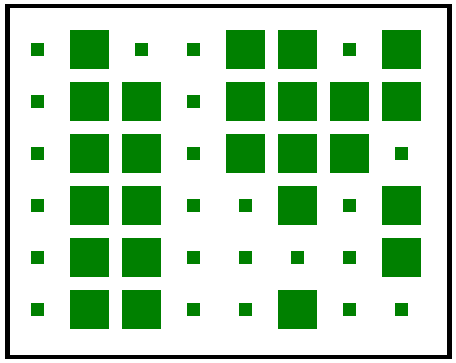
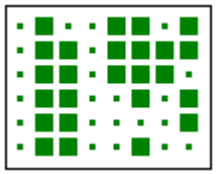


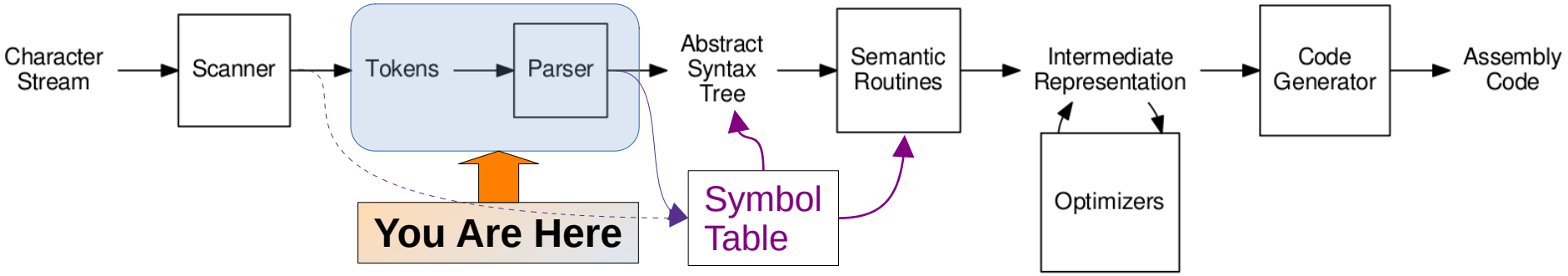
Class 10
Syntax Analysis: Parsing

CS 330
Programming Languages
And Compilers

Compilation Process Overview

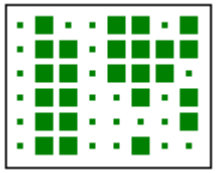


Thain, D. (2020). Introduction to compilers and language design.

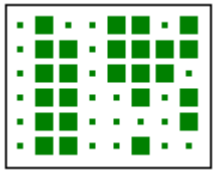


- Scanner (Lexical analyzer) – tokenizes
- **Parser (Syntax analyzer) – lexical units syntax check**
  - Creates symbol table and AST
- Semantic Routines – static semantics analysis/check
- Intermediate Rep – lower level language (optional)
- Optimizer – performance improvements (optional)
- Code Generator – outputs assembly lang (in our case)

Syntax Analysis Involves Parsing

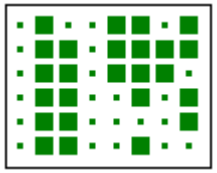


- Before we get into parsing, we need to recall several concepts that we've discussed
- Pop Quiz
  - What's the input, purpose, and output of the lexical analyzer?
  - What's the input, purpose, output of a parser?
  - What is a language generator and recognizer?
  - What is a grammar?
  - What makes up a grammar definition?
  - Explain leftmost and rightmost derivations
  - What is a sentential form?



Parse Trees

Grammar for Simple Language



(page 117 in Sebesta book)

Start Symbol

<assign> → <id> = <expr>

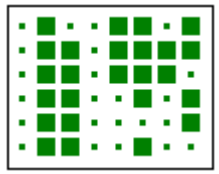
<id> → A | B | C

<expr> → <id> + <expr>
| <id> * <expr>
| (<expr>)
| <id>

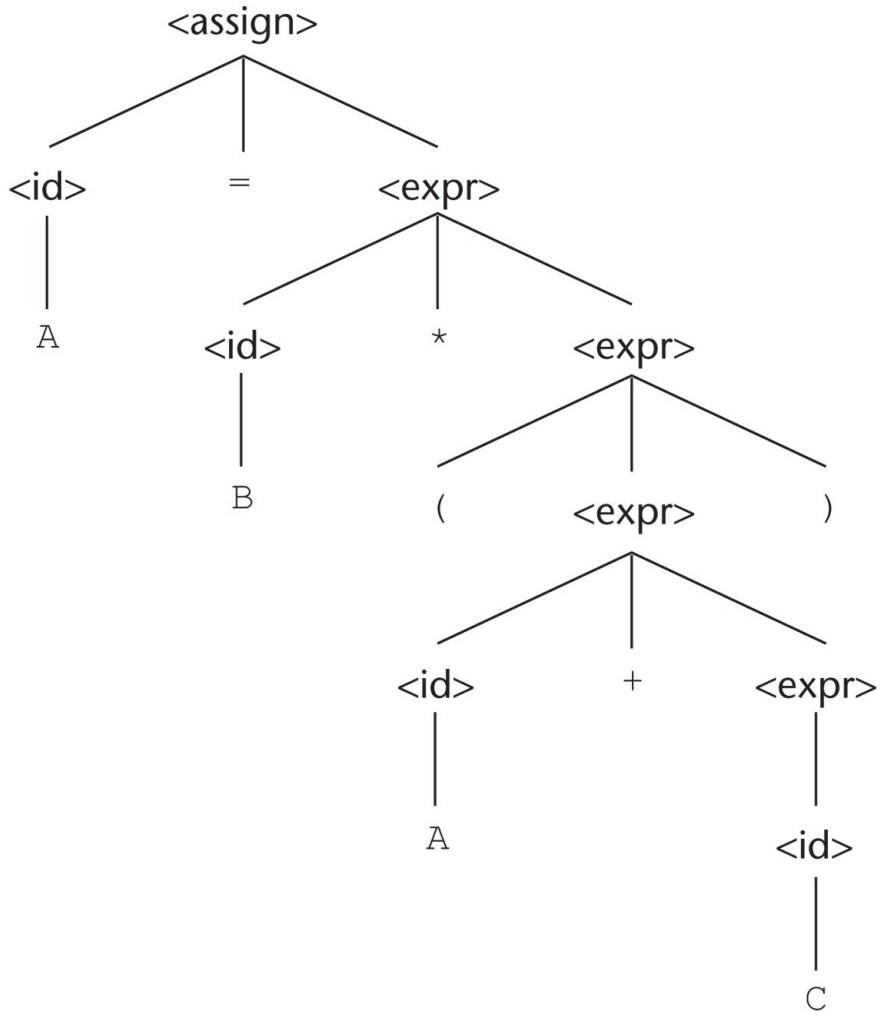
Previously we derived the statement

A = B * (A + C)

The Hierarchy of Sentences

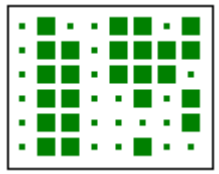


- Parse trees depict the structure of a sentence as a hierarchy
- Can use to understand the precedence of operations within a statement
- Here is the parse tree for the previous statement:
  `A = B * (A + C)`

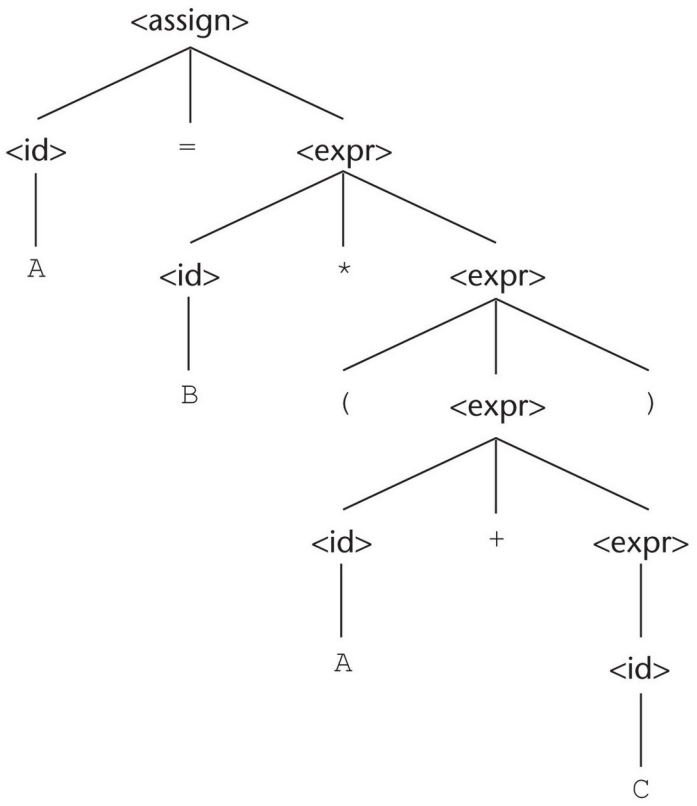


Copyright ©2016 Pearson Education, All Rights Reserved

Understanding a Parse Tree

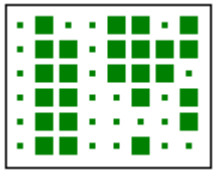


- What are at the leaves?
- What are at the internal nodes?
- What is at the root?
- Does a parse tree imply a particular order of derivation (e.g. leftmost, rightmost…)?



Copyright ©2016 Pearson Education, All Rights Reserved

Ambiguity in a Grammar



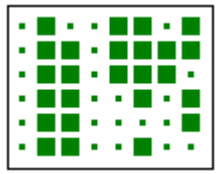
- If grammar allows a sentence to result in multiple hierarchies, it is ambiguous
  - This is problematic and must be avoided
- Here is a grammar with an ambiguity, the parse tree may grow on the left and right

```
<assign> → <id> = <expr>

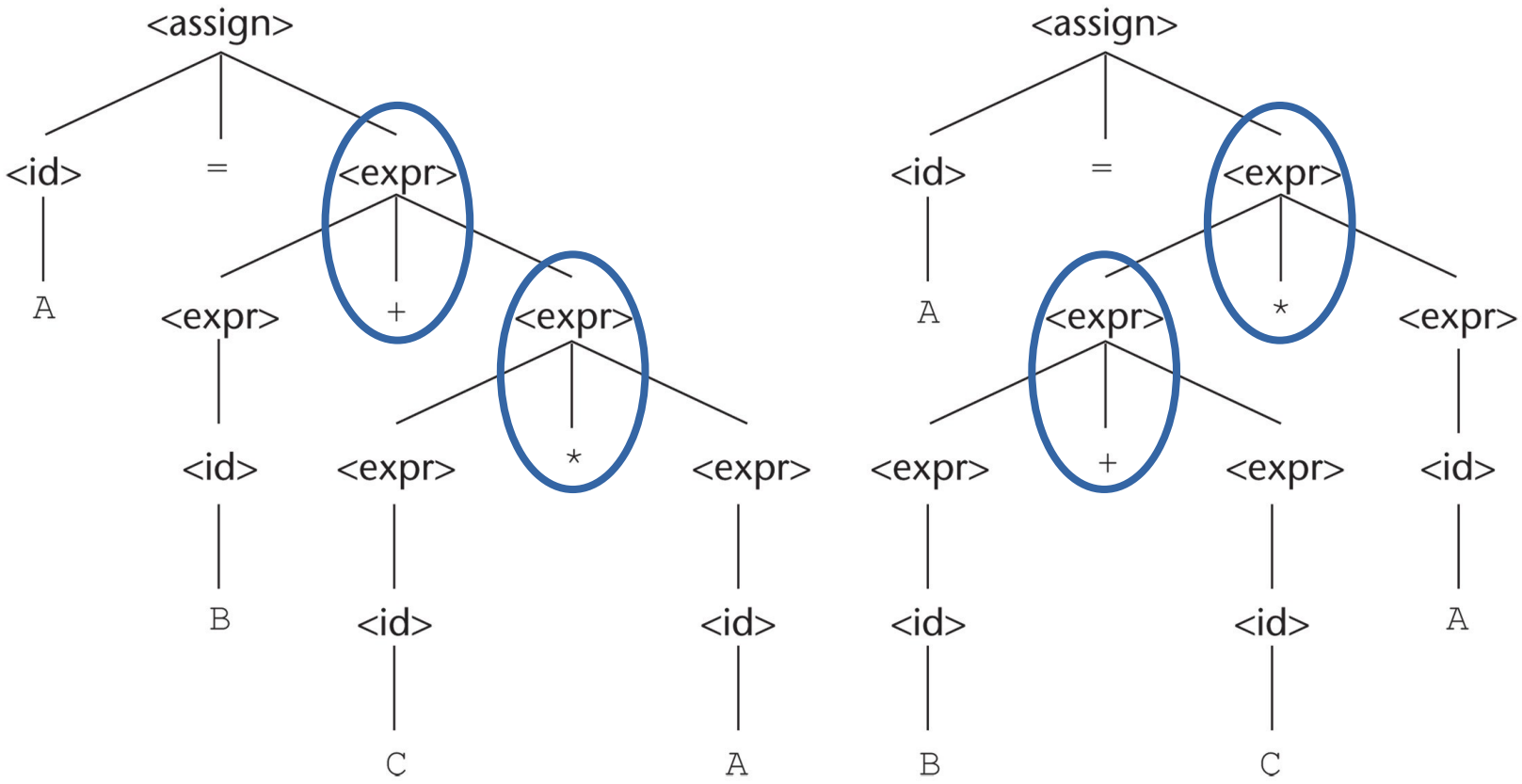
<id> → A | B | C

<expr> →  <expr> + <expr>
        |  <expr> * <expr>
        |  ( <expr> )
        |  <id>
```

Ambiguous Grammar Depiction

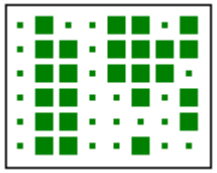


- Using the previous grammar, show parse trees that could result for `A = B + C * A`



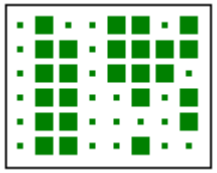
Copyright ©2016 Pearson Education, All Rights Reserved

Addressing Ambiguity



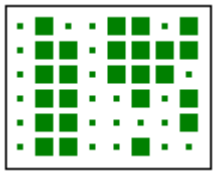
- To remove ambiguities in a grammar new non-terminals are added and changes impacting the definitions of existing non-terminals are made
  - We'll look at an "if-else" example of this process shortly

Operator Precedence



- The hierarchical structure of a parse tree implies operator precedence
  - Subtrees represent operations with higher precedence
    - They will be processed earlier and their results provided to the node above
- The grammar may be defined to assign different precedence to different operators
  - Allows the grammar to represent PEMDAS, for example

Grammar With Operator Precedence



- Defines multiplication to be at a higher precedence than addition

Start Symbol

<assign> → <id> = <expr>

<id> → A | B | C

<expr> → <expr> + <term>
| <term>

<term> → <term> * <factor>
| <factor>

<factor> → (<expr>)
| <id>

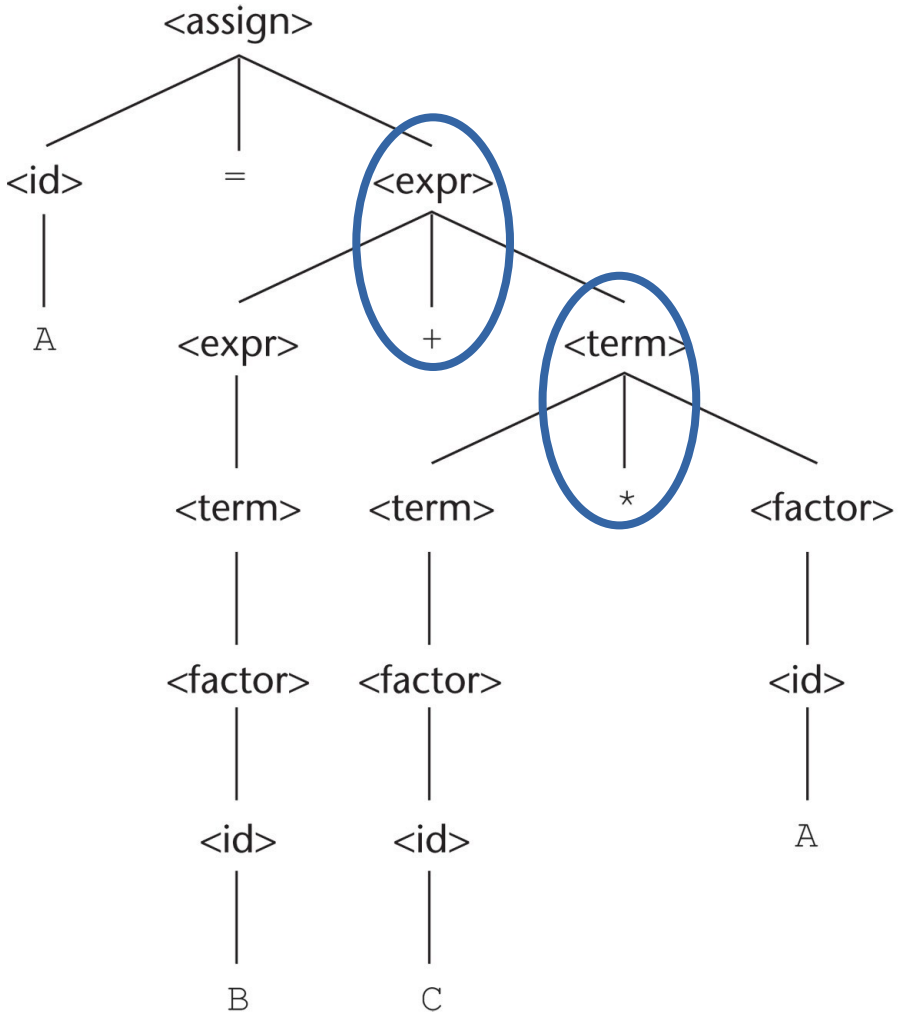
- Let's leftmost derive A = B + C * A

Resulting Parse Tree

- Parse tree for the previous expression and grammar

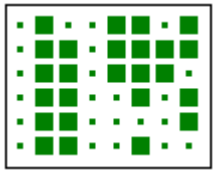
  A = B + C * A

- Note the fact that the multiplication ends up at a lower level and hence will be given higher precedence



Copyright ©2016 Pearson Education, All Rights Reserved

Left and Right Associative Operations



- Specify operations as left or right associative
- LHS *begins* the RHS, rule is **left recursive**, defines operation as **left associative**

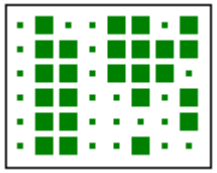
  <expr> → <expr> + <term>

- LHS *ends* the RHS the rule is **right recursive**, defines the operation to be **right associative**

  <factor> → <exp> ** <factor>
  | <exp>

  <exp> →(<expr>)
  | <id>

Java `if-else` Grammar Challenge



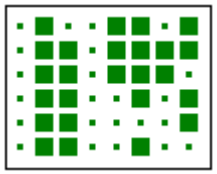
Given a partial grammar:

```
<if_stmt> → if (<logic_expr>) <stmt>
<if_stmt> → if (<logic_expr>) <stmt> else <stmt>
<stmt> → <assign>
<stmt> → <if_stmt>
```

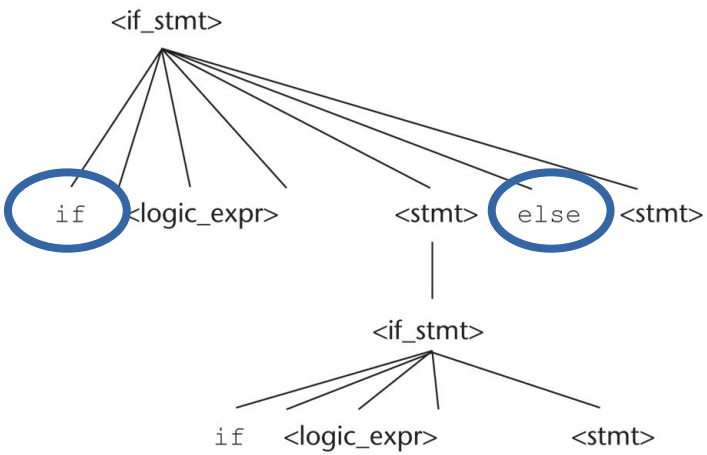
- What is an ambiguity in `if` statement's grammar?
- Consider the following program code:

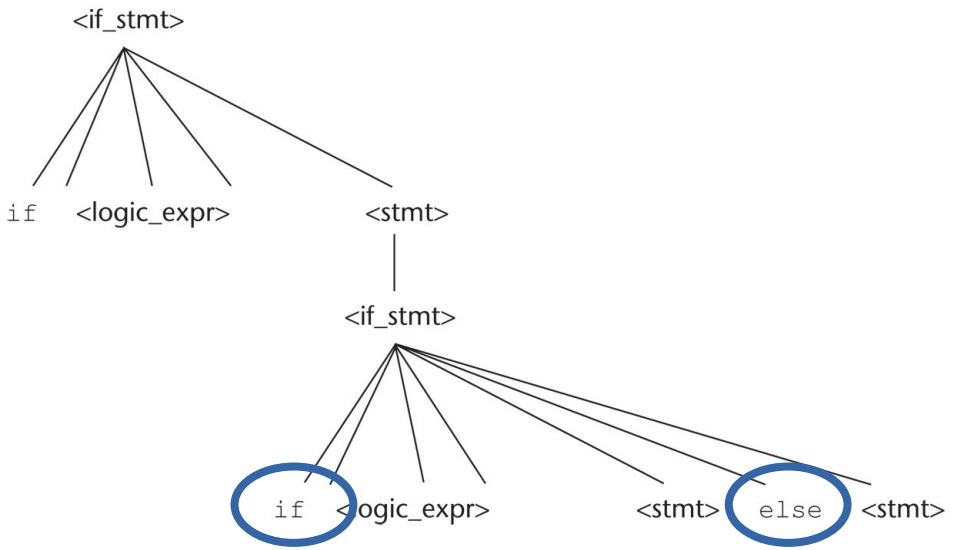
```
if (done == true)
if (denom == 0)
 quotent = 0;
else
    quotent = num / denom;
```

Two Possible if-else Parse Trees



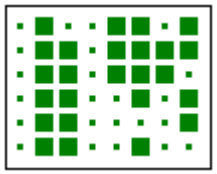
```
if (done == true)
if (denom == 0)
 quotient = 0;
else
    quotient =
     num / denom;
```





Copyright ©2016 Pearson Education, All Rights Reserved

Java if-else Grammar Fixed

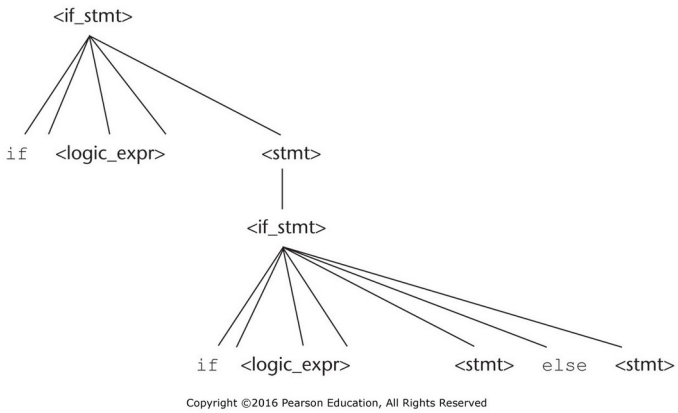


- Here is an unambiguous grammar for if-else:

```
<stmt> → <matched> | <unmatched>
<matched> →  if (<logic_expr>) <matched> else <matched>
           |   <any_non_if_stmt>
<unmatched> →   if (<logic_expr>) <stmt>
           |   if (<logic_expr>) <matched> else <unmatched>
```

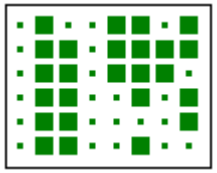
- There no ambiguity in the grammar now with respect to the earlier construct:

```
if (done == true)
if (denom == 0)
 quotient = 0;
else quotient = num / denom;
```



Copyright ©2016 Pearson Education, All Rights Reserved

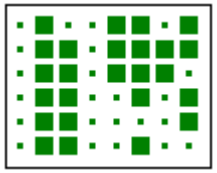
- Why?



Language Recognition

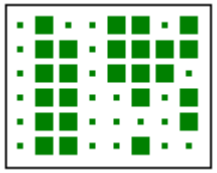
Syntax Analysis, a.k.a. Parsing

Overview of Syntax Analysis



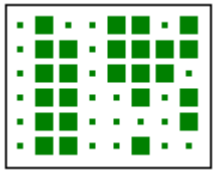
- Syntax analysis is the “parsing” step
  - Includes (conceptually) derivation of statements and creation of the parse tree
- Goals
  - Assess correctness of syntax
    - Alert user to errors, attempt to continue compilation. Why? Is this hard to do?
  - Create a parse tree or facsimile (trace) for intermediate code generation
    - Includes updates to the symbol table

Syntax Parsing Complexity



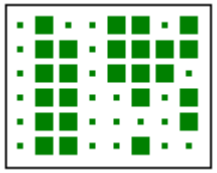
- Task is to take the lexical analyzer's output and trace the sentential form for the program
- Generalized parser that can parse any unambiguous syntax has complexity of $O(n^3)$
  - Complexity due to rework of finding the correct sentential form – may have to go back several steps if wrong form is chosen
- For efficiency we design programming languages to reduce generality and complexity
  - Programming language syntax parser [LL/LR] has complexity of $O(n)$

Two Categories of Parsers: Top-Down

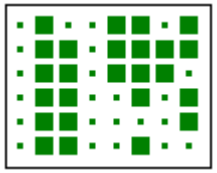


- Easily represented as a tree
  - Preorder traversal begins at the root and each node is visited before following its branches
  - Branches are followed in left-to-right order
- Results in a leftmost derivation

Two Categories of Parsers: Bottom-Up

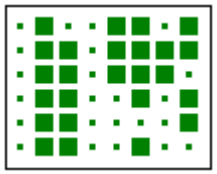


- Builds the parse tree starting at the leaves and moving to the root
- Results in the reverse of a rightmost derivation (arrived at backwards)



Top-down Parsing

Review: Grammar and Derivation



(page 117 in Sebesta book)

Start Symbol

<assign> → <id> = <expr>

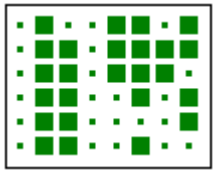
<id> → A | B | C

<expr> → <id> + <expr>
| <id> * <expr>
| (<expr>)
| <id>

Use leftmost derivation to derive A = B * (A + C)

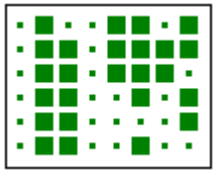
Draw the corresponding parse tree

Top-down Parser Overview



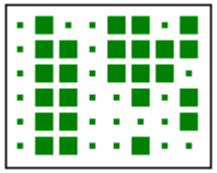
- Like our manual derivation work, the top-down parser identifies the next sentential form for the leftmost derivation
  - Find the correct grammar rule from the language definition
  - Take current form, check the next token, and choose the RHS that matches that token in the left-most location

Top-down Parser Operation



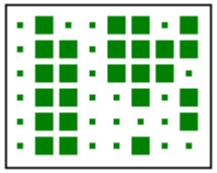
- As input we have a possible sentence (program) in the language that may or may not be syntactically correct
- The lexical analyzer is called by the parser to get the next token (and associated lexeme if needed) from the input sentence (program)
- Parser arrives at next sentential form and again asks the lexical parser for a token
- This continues until the lexical analyzer runs out of tokens (end of program)

A Step in the Life of a Top Down Parser



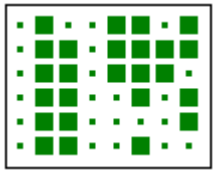
- Consider step in the top down parser where we have a sentential form that is part of a leftmost derivation, the goal is to find the next sentential form in that leftmost derivation
- e.g. Given a left sentential form xAα
  - where x is a string of terminals
  - A is a non-terminal (left-most)
  - α is a string of terminals and non-terminals
- Since we want a leftmost derivation, what would we do next?

A Step in the Life of a Top Down Parser



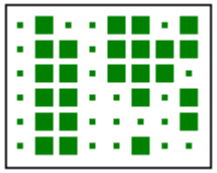
- We have a left sentential form xAα
- How should A (leftmost non-terminal) be expanded?
  - Get the next token from lexical analyzer
  - Determine production to expand from token
  - For “one lookahead” grammar, only one RHS of A will have the token as its first symbol
    - If there is no RHS of A with its first symbol being the token returned, what do you think occurred?

Top-down Parser Implementation



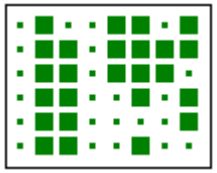
- Classic top-down parser design uses recursion: a recursive-descent parser
  - Written as an implementation of the BNF description of the syntax (first/follow sets)
- Can be represented by a parsing table containing the BNF rules in table form
- These top down parsers are LL parsers
  - L for left-to-right scan of the input
  - L for leftmost derivation

LL Grammar Class



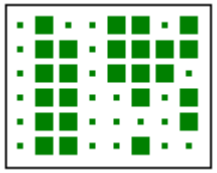
- LL parsers designated with number *k*, as *LL(k)*
  - *k* is number of look-ahead tokens used to determine the production to use when expanding the leftmost non-terminal
- LL(1) parsers work in many cases but have an Achilles heel: if the grammar includes **direct left recursion** the parser will fail
  - Example: `<term>` → `<term> + <var>`
    - Issue is that this will lead to "infinite recursion" in the implementation since `term()` will initially call `term()`…

Addressing Direct Left Recursion



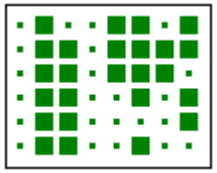
- Group all RHS for non-terminal containing a direct left recursion RHS such that those that include direct left recursion are first
- Replace RHS containing direct left recursion with two rules
  - Define non-terminal with non-direct left recursion RHS option and add new non-terminal to the end
  - Define new non-terminal with first terminal or non-terminal following the original non-terminal (one with direct left recursion)

Address Direct Left Recursion 1/6



<expr>→ <expr> + <factor> | <factor>
<factor>→ <factor> * <val> | <val>
<val>→ (<expr>) | id

Address Direct Left Recursion 2/6

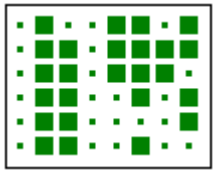


<expr> → <expr> + <factor> | <factor>

<factor> → <factor> * <val> | <val>

<val> → (<expr>) | id

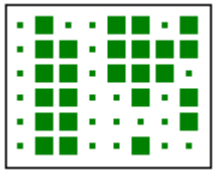
Address Direct Left Recursion 3/6



<expr>→ <expr> + <factor> | <factor>
<factor>→ <factor> * <val> | <val>
<val>→ (<expr>) | id

- “fixed” <expr> rules
  <expr>→ <factor> <expr_2>
  <expr_2>→ + <factor> <expr_2> | Є

Address Direct Left Recursion 4/6



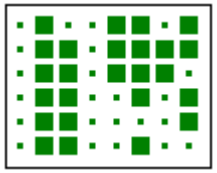
<expr> → <expr> + <factor> | <factor>
<factor> → <factor> * <val> | <val>
<val> → (<expr>) | id

- "fixed" <expr> rules

  <expr> → <factor> <expr_2>

  <expr_2> → + <factor> <expr_2> | ϵ

Address Direct Left Recursion 5/6



<expr> → <expr> + <factor> | <factor>
<factor> → <factor> * <val> | <val>
<val> → (<expr>) | id

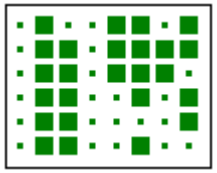
- "fixed" <expr> rules

  <expr> → <factor> <expr_2>

  <expr_2> → + <factor> <expr_2> | ϵ

- ϵ is the **erasure rule**
  - Erases LHS from the sentential form

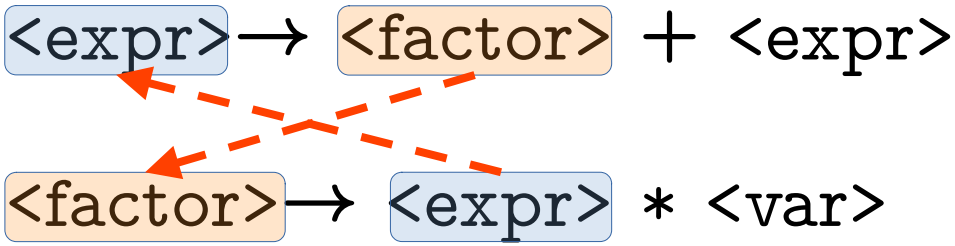
Address Direct Left Recursion 6/6



<expr> → <expr> + <factor> | <factor>
<factor> → <factor> * <val> | <val>
<val> → (<expr>) | id

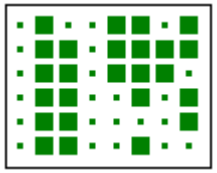
- "fixed" <expr> rules
  - <expr> → <factor> <expr_2>
  - <expr_2> → + <factor> <expr_2> | ϵ
- "fixed: <factor> rules
  - <factor> → <val> <factor_2>
  - <factor_2> → * <val> <factor_2> | ϵ
- ϵ is the **erasure rule**
  - Erases LHS from the sentential form

Indirect Left Recursion



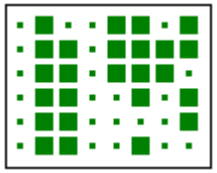
- Implementation will have `expr()` call `factor()`, which calls `expr()`…
- Can be solved through grammar refactoring
- Easiest approach is to avoid direct and indirect left recursion in the grammar when initially building the specification, if possible
  - Otherwise, we need a more robust parser

All We Have Are Tokens



- The syntax parser's view of our code is a sequence of tokens
- LL(1) parser has access to its current state and the next token only
- It cannot "look ahead" to see what is coming
  - Must be able to determine correct RHS from that one token
- If grammar has multiple RHS for LHS, first terminal in each RHS must be unique to LHS
  - RHS must be **pairwise disjoint**

Memory Lane: Is This Grammar LL(1)?



Use leftmost derivation to derive: A=B+C; B=C

Start Symbol

<program> → **begin** <stmt_list> **end**

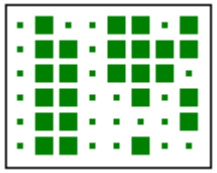
<stmt_list> → <stmt>
| <stmt>; <stmt_list>

<stmt> → <var> = <expression>

<var> → **A** | **B** | **C**

<expression> → <var> + <var>
| <var> - <var>
| <var>

Pairwise Disjoint and Left Factoring



- If grammar has terminal which is not pairwise disjoint can be addressed using left factoring

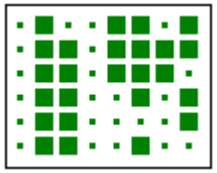
  <var>→ id | id[<expr>]

  - Tokens arriving will be IDENT
  - or IDENT OBRACKET
- Add new non-terminal to remove the repeating terminal (id) in the RHS and use erasure rule to optionally include next token

  <var>→ id <new>

  <new>→ ϵ | [<expr>]

Pairwise Disjointness Test

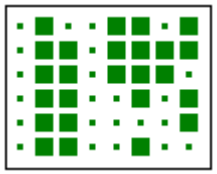


- Pairwise disjointness test to determine if a grammar has this limitation
  - Determine the first terminal for every RHS of a production with the same LHS, the first terminal must be different for each

```
<A> -> a <B> | b<A>b | c <C> | d
```

  - The 4 RHSs of `A` each have as their first symbol `{a}`, `{b}`, `{c}`, `{d}` respectively
  - These are all disjoint and pass the test. Ones on previous slide are not pairwise disjoint and do not (requiring refactoring)

Pairwise Disjointness Test Example



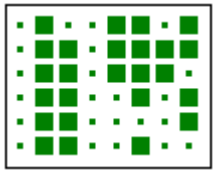
- If first symbol in the RHS is a non-terminal, need to expand it to identify the first terminal
- e.g. `<A> -> <B> a | b<A>b | c <C> | d`

  `<B> -> <C> f`

  `<C> -> s <F>`

- Find the first terminal of `A` by following `<B>` then `<C>` to find out that the terminal is `s`
- The 4 RHS of `A` have as their first symbols `{s}`, `{b}`, `{c}`, `{d}` respectively
- Sets are disjoint so the productions for `<A>` would pass the test

Homework



- Monday (10/5)
  - Lab 2 due (team and individual)
- Wednesday (10/7)
  - Compilers book Appendix B



- Friday (10/9)
  - LLM: Yann LeCun Has Been Right...



- Sunday (10/11)
  - Journal entry for week 5
    - **2-1 Rule** (minimum 2 paragraphs, 1 quiz question)
- Wednesday (10/14)
  - **Lab 3 Due** (B-minor Scanner)